

PROYECTO FINAL - UT7.RA7

GESTIÓN DE INFORMACIÓN EN BASES DE DATOS NO RELACIONALES

Desarrollo e implementación práctica con MongoDB Atlas y Compass

MÓDULO

Bases de Datos

RESULTADO DE APRENDIZAJE

RA7 – Gestión de información en BD no relacionales

AUTORES

Oliver Bitica

Rubén Barrado Pastor

CURSO & FECHA

1º DAM (2025/26)

Mayo de 2026

Índice General

1	Introducción	3
2	Bloque A – Caracterización de las bases de datos no relacionales	3
2.1	Qué es una base de datos no relacional	3
2.2	Características principales	3
2.3	Comparativa básica: Relacional vs No relacional	4
2.4	Ventajas e inconvenientes	5
3	Bloque B – Tipos principales de bases de datos no relacionales	5
3.1	Clave-valor (Key-Value)	5
3.2	Documental	5
3.3	Columnar / Familias de columnas	6
3.4	Grafos	6
3.5	Resumen rápido	6
4	Bloque C – Elementos utilizados en estas bases de datos	7
4.1	Equivalencias entre mundos	7
4.2	Conceptos clave en MongoDB	7
4.3	Ejemplo de documento real (de la base sample_mflix.movies)	8
5	Bloque D – Formas de gestión de la información según el tipo	8
5.1	Cómo se gestiona la información en MongoDB	9
6	Bloque E – Práctica con MongoDB Atlas + Compass	9
6.1	Conexión al clúster con Compass	9
6.2	Exploración de las bases de datos de muestra	10
6.3	Consultas (READ) con filtros en Compass	11
6.4	Creación de una base de datos propia para escritura	14
6.5	Inserción de documentos (CREATE)	15
6.6	Actualización (UPDATE) y borrado (DELETE) desde Compass	16
6.7	Creación de Índices	18
6.8	Pipeline de Agregación (Consulta avanzada)	19
6.9	Exportación e importación de datos	21
6.10	Validación de esquemas	23
7	Conclusiones	26
8	Reparto de tareas	27
9	Bibliografía	27

1 Introducción

En lo que llevamos de curso de Bases de Datos en 1º de DAM, nos hemos centrado sobre todo en el modelo relacional (usando MySQL): bases de datos estructuradas en tablas fijas, filas, columnas, claves primarias y foráneas, y haciendo consultas con SQL. Sin embargo, en el mundo real muchas aplicaciones manejan datos masivos, muy cambiantes o directamente desordenados. Para solucionar esto nacieron las bases de datos **no relacionales** (también llamadas NoSQL).

En esta memoria vamos a explicar de forma sencilla y directa qué son estas bases de datos, qué tipos existen, qué componentes las forman y cómo se gestionan.

Para la parte práctica, hemos trabajado con **MongoDB**, que es la base de datos documental más popular. Hemos usado un clúster en la nube con **MongoDB Atlas** (en su capa gratuita M0 en AWS Fráncfort) y nos hemos conectado desde nuestros ordenadores con **MongoDB Compass**, que es la herramienta gráfica oficial. La cadena de conexión compartida que hemos usado es:

```
mongodb+srv://<usuario>:<password>@dam.pb0dxmb.mongodb.net/
```

Usuario: **admin** Contraseña: 123

Para no empezar de cero y poder probar consultas potentes desde el primer minuto, el clúster ya venía cargado con varias **bases de datos de ejemplo** que ofrece MongoDB (como `sample_mflix` para películas, `sample_airbnb` para alojamientos, etc.). Esto nos ha permitido jugar con datos reales sin tener que importar nada nosotros al principio.

2 Bloque A – Caracterización de las bases de datos no relacionales

2.1 Qué es una base de datos no relacional

Una base de datos no relacional (NoSQL) es aquella que **no almacena los datos en las clásicas tablas de filas y columnas**. En su lugar, utiliza formas mucho más libres y flexibles para guardar la información: documentos JSON, parejas de clave-valor, árboles de nodos (grafos) o familias de columnas.

Surgieron a principios de los años 2000 porque las grandes aplicaciones web (como Google, Amazon o Facebook) necesitaban guardar millones de datos al segundo y el modelo relacional tradicional con tablas rígidas y consultas complejas se quedaba demasiado lento y costoso.

2.2 Características principales

- **Sin esquema rígido:** A diferencia de MySQL, donde todas las filas de una tabla deben tener exactamente las mismas columnas, aquí cada registro puede ser diferente. Si mañana

queremos añadir un campo nuevo a un único registro, lo hacemos y listo, sin tener que alterar toda la base de datos.

- **Escalabilidad horizontal (Sharding):** Si tu base de datos se queda pequeña, en vez de comprar un servidor gigante y súper caro (escalado vertical), añades varios ordenadores normales y baratos en red y repartes los datos entre ellos.
- **Mucha rapidez de lectura y escritura:** Al no tener que comprobar relaciones complejas entre tablas cada vez que escribes, la base de datos funciona a toda velocidad.
- **Muy cómodo para programar:** Los datos se guardan en formatos muy parecidos a los objetos que usamos al programar en Java o en JavaScript, por lo que no hace falta hacer traducciones raras de código a tablas.

Teorema CAP y Consistencia Eventual (BASE)

- El **Teorema CAP** dice que en un sistema distribuido (repartido en varios servidores) es imposible tener a la vez el 100% de estas tres cosas: **Consistencia** (que todos los servidores tengan la misma información al instante), **Autonomía/Disponibilidad** (que el sistema siempre responda aunque algún servidor falle) y **Partición** (tolerancia a fallos de red). Como internet siempre puede fallar, tenemos que elegir entre Consistencia y Disponibilidad.
- Por eso muchas NoSQL prefieren cumplir **BASE** en lugar de las reglas estrictas de ACID. Esto significa que el sistema prefiere estar siempre disponible y rápido, aceptando una **consistencia eventual**: los datos tardarán unos segundos en actualizarse en todos los ordenadores de la red, pero al final todos acabarán teniendo lo mismo.

2.3 Comparativa básica: Relacional vs No relacional

Característica	Relacional (MySQL / SQL)	No relacional (NoSQL)
Estructura	Tablas de filas y columnas fijas.	Documentos JSON, clave-valor, grafos...
Esquema	Estricto. Hay que definirlo antes de meter datos.	Flexible. Cada registro puede ser diferente.
Lenguaje	SQL estándar.	Cada base de datos tiene sus comandos (API propia).
Escalado	Vertical (comprar un servidor mejor y más caro).	Horizontal (añadir más ordenadores baratos a la red).
Relaciones	Claves ajenas y operaciones JOIN costosas.	Documentos dentro de otros o referencias simples.
Garantías	ACID fuerte (consistencia inmediata asegurada).	BASE (consistencia eventual para ganar velocidad).

Mejor para	Cosas críticas como bancos, facturas, ERPs.	Apps móviles, Big Data, redes sociales, catálogos.
-------------------	---	--

2.4 Ventajas e inconvenientes

- **Ventajas:** Es facilísimo cambiar el diseño de los datos sobre la marcha, escala muy bien de forma barata y es súper rápida.
- **Inconvenientes:** No hay un lenguaje estándar único como SQL (si cambias de base de datos tienes que aprender comandos nuevos), no es fácil hacer búsquedas complejas que relacionen muchas colecciones y tienes que controlar tú en tu programa que no se metan datos absurdos.

3 Bloque B – Tipos principales de bases de datos no relacionales

Las bases de datos NoSQL se agrupan en cuatro familias principales según cómo guardan los datos:

3.1 Clave-valor (Key-Value)

- **Cómo funciona:** Es el modelo más simple. Guarda la información en parejas formadas por una clave única (un ID) y un valor (que puede ser cualquier cosa: un texto, un número o una lista). Es exactamente igual que un diccionario de Python o un Map de Java.
- **Puntos fuertes/débiles:** Es increíblemente rápida para leer y escribir / No puedes buscar cosas por el contenido del valor, solo puedes buscar por la clave.
- **Ejemplos:** Redis, Amazon DynamoDB.
- **Uso típico:** Guardar carritos de la compra de usuarios, sesiones activas o cachés de páginas web.

3.2 Documental

- **Cómo funciona:** Es una evolución de la clave-valor. En lugar de valores sueltos, guarda **documentos** (normalmente ficheros en formato JSON). Cada documento tiene pares de campo: valor.
- **Puntos fuertes/débiles:** Súper flexible y permite hacer búsquedas por cualquier campo que esté dentro del documento / Si no vigilamos, podemos acabar duplicando mucha información.
- **Ejemplos:** **MongoDB** (la que usamos en la práctica), CouchDB, Firebase Firestore.
- **Uso típico:** Páginas de blogs, tiendas online con catálogos de productos muy variados o APIs que intercambian JSONs.

3.3 Columnar / Familias de columnas

- **Cómo funciona:** En lugar de guardar los datos fila a fila, los guarda por columnas. Esto permite leer columnas enteras a la vez.
- **Puntos fuertes/débiles:** Ideal para procesar estadísticas sobre millones de registros / Modelar la base de datos es difícil y no sirve para consultar registros individuales de forma rápida.
- **Ejemplos:** Cassandra, HBase.
- **Uso típico:** Registro de sensores (IoT), análisis de Big Data, almacenamiento de logs del sistema.

3.4 Grafos

- **Cómo funciona:** Representa los datos usando **nodos** (entidades, como personas) y **relaciones/aristas** (las líneas que los unen, como “es amigo de”, “ha comprado”).
- **Puntos fuertes/débiles:** Te permite navegar por redes de relaciones complejas en milisegundos / No sirve para almacenar listas de datos planos o simples.
- **Ejemplos:** Neo4j, Amazon Neptune.
- **Uso típico:** Redes sociales (Facebook, LinkedIn), motores de recomendación (Netflix, Spotify) o sistemas para detectar fraude con tarjetas de crédito.

3.5 Resumen rápido

Tipo	Formato de almacenamiento	Ejemplo	Lo mejor para
Clave-valor	clave -> valor	Redis	Cachés muy rápidas y carritos
Documental	Documentos JSON	MongoDB	Datos variados y APIs web
Columnar	Columnas agrupadas	Cassandra	Big Data y analizar logs
Grafos	Nodos y líneas de conexión	Neo4j	Redes sociales y recomendaciones

Por qué elegimos MongoDB

Al ser de tipo documental, es la más versátil y fácil de entender si vienes de SQL. Además, tiene herramientas visuales muy cómodas (Compass) y una nube gratis muy fácil de usar (Atlas).

4 Bloque C – Elementos utilizados en estas bases de datos

Para no perdernos al usar MongoDB, podemos comparar sus elementos con los que ya conocemos de MySQL:

4.1 Equivalencias entre mundos

Modelo Relacional (MySQL)	Modelo Documental (MongoDB)
Base de datos	Base de datos
Tabla	Colección
Fila / Registro	Documento
Columna / Campo	Campo
Clave Primaria (PK)	Campo <code>_id</code>
Índice	Índice
Operación JOIN	Documentos anidados (subdocumentos) o <code>\$lookup</code>

4.2 Conceptos clave en MongoDB

- **Documento:** La pieza básica donde guardamos la información. Es un objeto con formato JSON que contiene parejas de campo: valor.
- **Colección:** Un grupo de documentos. Equivale a una tabla de MySQL, pero con la diferencia de que los documentos dentro de una colección no tienen por qué ser idénticos.
- **Base de datos:** El contenedor general que guarda las colecciones.
- **BSON:** Es el formato que usa MongoDB por dentro para guardar la información. Es un "JSON Binario". Lo usa porque al ordenador le resulta mucho más rápido procesarlo y porque admite más tipos de datos que el JSON básico (como fechas reales, números enteros, decimales precisos o IDs especiales).
- **Campo `_id`:** Es el DNI obligatorio de cada documento. Funciona como la clave primaria. Si al insertar un documento no le pones este campo, MongoDB se lo inventa automáticamente y le asigna un código único gigante llamado `objectId`.
- **Tipos de datos:** Texto (string), números (enteros y decimales), booleanos (true/false), fechas, valores nulos (null), listas (arrays) y otros documentos metidos dentro (subdocumentos).
- **Documentos anidados y Arrays:** Una de las mejores cosas de MongoDB. En lugar de hacer una tabla separada para guardar los teléfonos de un cliente y tener que hacer un JOIN, metemos una lista ["123456", "987654"] directamente en un campo llamado `telefonos` dentro del propio documento del cliente.

- **Índices:** Estructuras de datos que se crean sobre campos muy consultados (como el nombre o el correo) para que las búsquedas vayan a toda velocidad en lugar de tener que leer toda la base de datos entera.
- **Replica Set (Réplica):** Copias idénticas de nuestra base de datos repartidas en otros servidores de forma automática. Si el servidor principal se rompe, uno de los secundarios se activa en milisegundos y la aplicación sigue funcionando sin que el usuario note nada.
- **Sharding:** Dividir los datos de una colección gigante y repartirlos entre varios servidores para poder almacenar más información de la que cabría en un solo disco duro.

4.3 Ejemplo de documento real (de la base `sample_mflix.movies`)

```
{
  "_id": ObjectId("573a1390f29313caabcd4135"),
  "title": "The Matrix",
  "year": 1999,
  "genres": ["Action", "Sci-Fi"],
  "runtime": 136,
  "cast": ["Keanu Reeves", "Laurence Fishburne", "Carrie-Anne Moss"],
  "imdb": { "rating": 8.7, "votes": 1500000 }
}
```

Aquí se ve todo muy claro: el `_id` autogenerado, campos de texto y número, listas (arrays) en `genres` y `cast`, y un subdocumento metido en el campo `imdb` para guardar la nota y los votos juntos.

5 Bloque D – Formas de gestión de la información según el tipo

Como en NoSQL no existe un lenguaje estándar como el SQL, cada familia tiene su propia manera de trabajar:

Tipo	Cómo se manejan los datos	Lenguaje o interfaz
Clave-valor (Redis)	Comandos directos para guardar y recuperar usando la clave.	Comandos como SET y GET
Documental (MongoDB)	Funciones CRUD para buscar, meter, cambiar y borrar.	MQL (MongoDB Query Language)
Columnar (Cassandra)	Consultas parecidas a SQL pero adaptadas a columnas.	CQL (Cassandra Query Language)
Grafos (Neo4j)	Dibujar patrones con flechas para buscar caminos de relación.	Cypher

5.1 Cómo se gestiona la información en MongoDB

En MongoDB trabajamos usando **MQL (MongoDB Query Language)** mediante las siguientes herramientas:

- **Operaciones CRUD:**
 - **Crear:** `insertOne` (meter un documento) o `insertMany` (meter una lista de documentos).
 - **Leer:** `find` (buscar documentos aplicando filtros).
 - **Actualizar:** `updateOne` o `updateMany` para modificar campos usando operadores como `\$set` o `\$inc` (para sumar).
 - **Borrar:** `deleteOne` o `deleteMany`.
- **Operadores de filtro:** Para buscar datos usamos operadores especiales como `\$gt` (mayor que), `\$lt` (menor que), `\$in` (que esté en una lista), `\$and`, `\$or` o `\$regex` (búsquedas por texto).
- **Framework de Agregación:** Es una de las herramientas más potentes de MongoDB. Funciona como una tubería (pipeline) en la que los documentos entran por un extremo y van pasando por diferentes “etapas” de procesamiento. Por ejemplo, primero filtramos con `\$match`, luego troceamos listas con `\$unwind`, después agrupamos para sumar o hacer medias con `\$group` y finalmente ordenamos el resultado con `\$sort`. Es el equivalente al `GROUP BY` y los `JOIN` complejos de SQL, pero mucho más visual y paso a paso.
- **Herramientas de importación y exportación:** Utilidades como `mongoimport` y `mongoexport` para mover datos rápidamente en formatos estándar como JSON o CSV.
- **Seguridad:** Creación de usuarios con contraseñas y asignación de roles (permisos) para controlar quién puede leer o escribir en cada colección.

6 Bloque E – Práctica con MongoDB Atlas + Compass

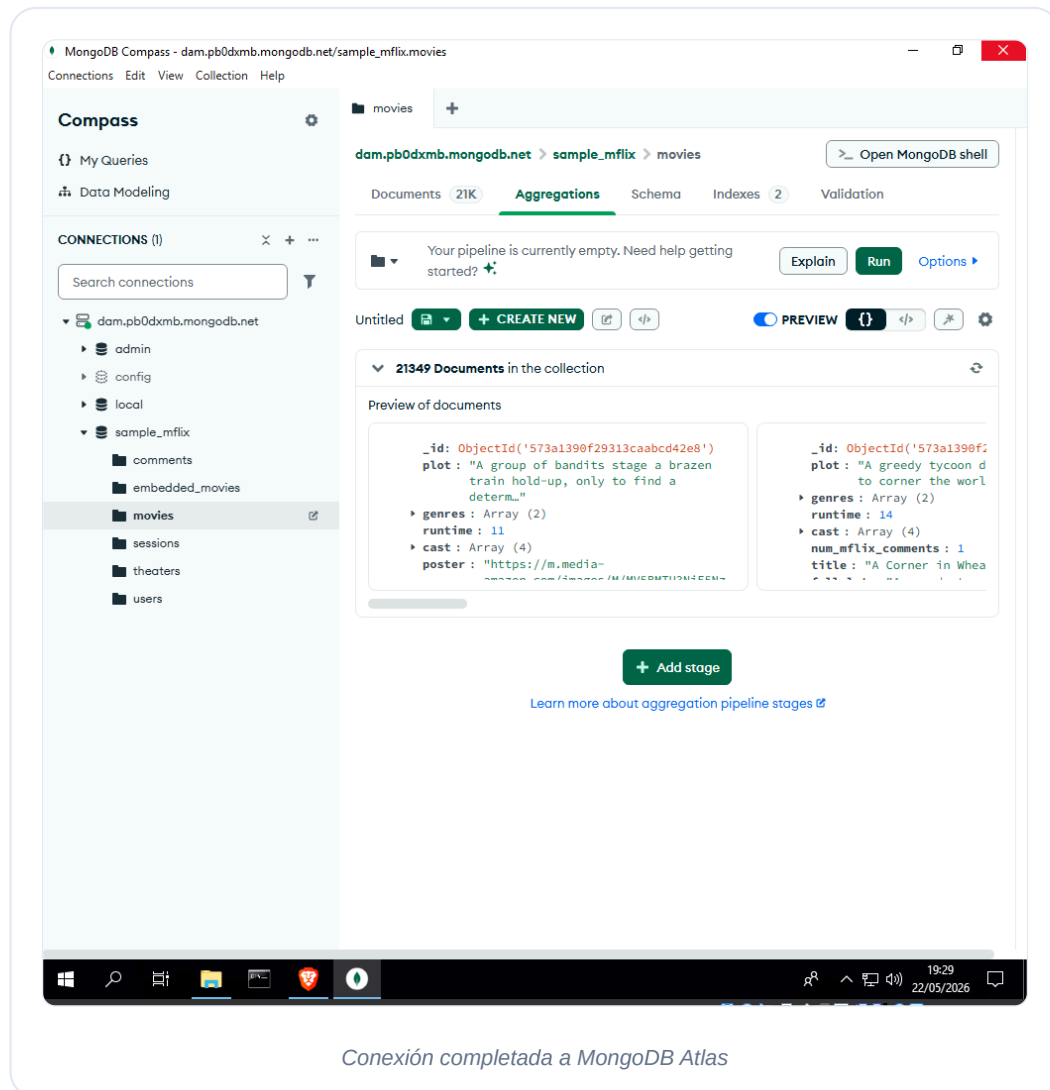
A continuación, mostramos los pasos prácticos que hemos realizado para aprender a utilizar MongoDB conectándonos al clúster compartido en la nube.

6.1 Conexión al clúster con Compass

Abrimos el programa de escritorio **MongoDB Compass**, pegamos la cadena de conexión compartida en la barra de direcciones (poniendo `admin` en el usuario y `123` en la contraseña) y pulsamos **Connect**.

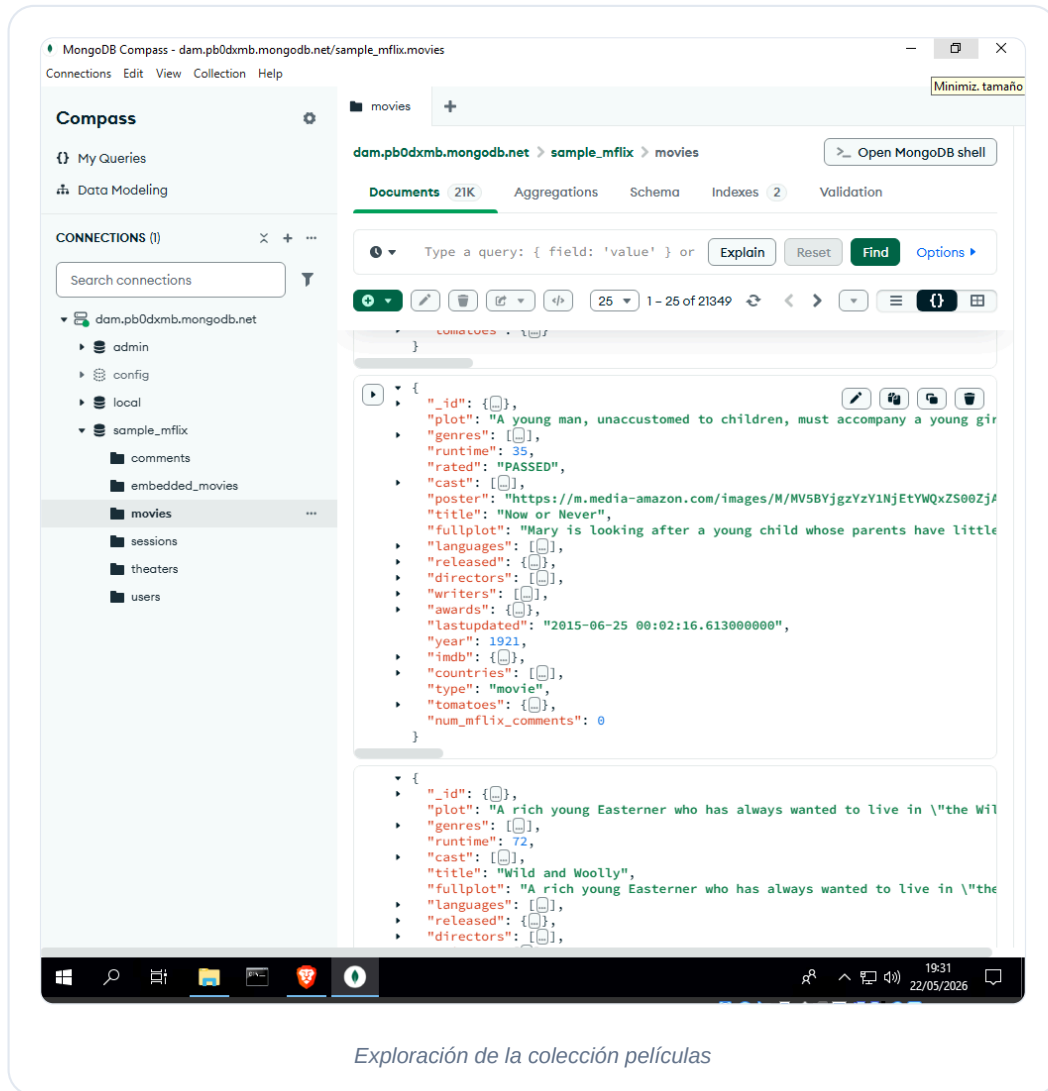
```
mongodb+srv://admin:123@dam.pb0dxmb.mongodb.net/
```

Compass se conecta y nos muestra en la barra lateral izquierda todas las bases de datos que tenemos disponibles en la nube.



6.2 Exploración de las bases de datos de muestra

Para practicar consultas de lectura, abrimos la base de datos `sample_mflix` y nos metemos en la colección `movies`. Esta colección contiene miles de películas reales con un montón de campos (título, año, notas de IMDb, géneros, actores, etc.), lo cual es fantástico para probar filtros realistas.



6.3 Consultas (READ) con filtros en Compass

Para filtrar las películas en Compass, escribimos los filtros en formato JSON en la barra de búsqueda de la pestaña **Documents** y pulsamos **Find**. Aquí están las consultas de prueba que realizamos:

- **C1: Películas del año 1999.**

```
{ year: 1999 }
```

- **C2: Películas con una nota en IMDb superior a 8.5.**

```
{ "imdb.rating": { "$gt": 8.5 } }
```

- **C3: Películas que sean del género “Comedy” (Comedia).** MongoDB busca automáticamente dentro del array de géneros.

```
{ genres: "Comedy" }
```

- **C4: Películas que sean de Acción (“Action”) o de Aventuras (“Adventure”).**

```
{ genres: { $in: ["Action", "Adventure"] } }
```

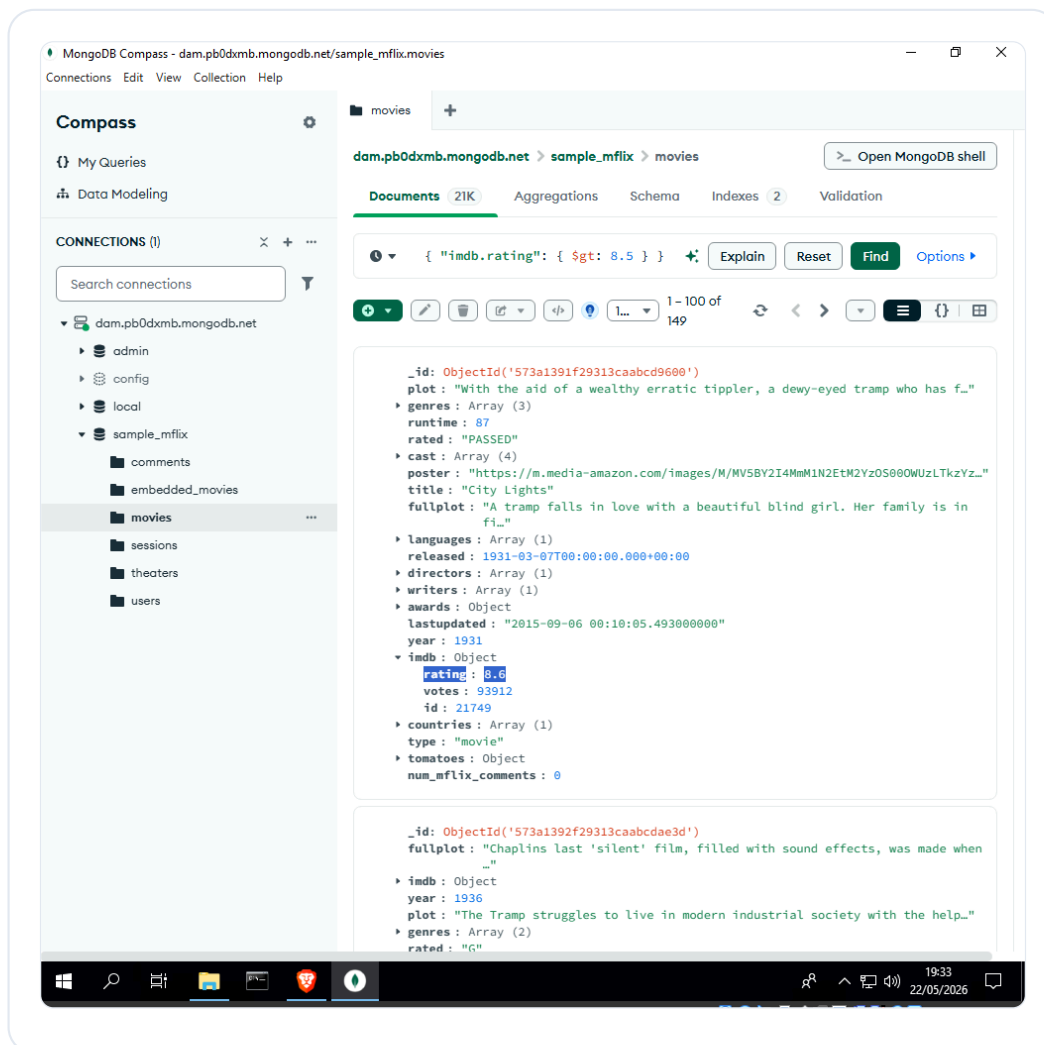
- **C5: Películas estrenadas entre los años 2000 y 2005 que tengan más de un 8 en IMDb.**

```
{ year: { $gte: 2000, $lte: 2005 }, "imdb.rating": { $gt: 8 } }
```

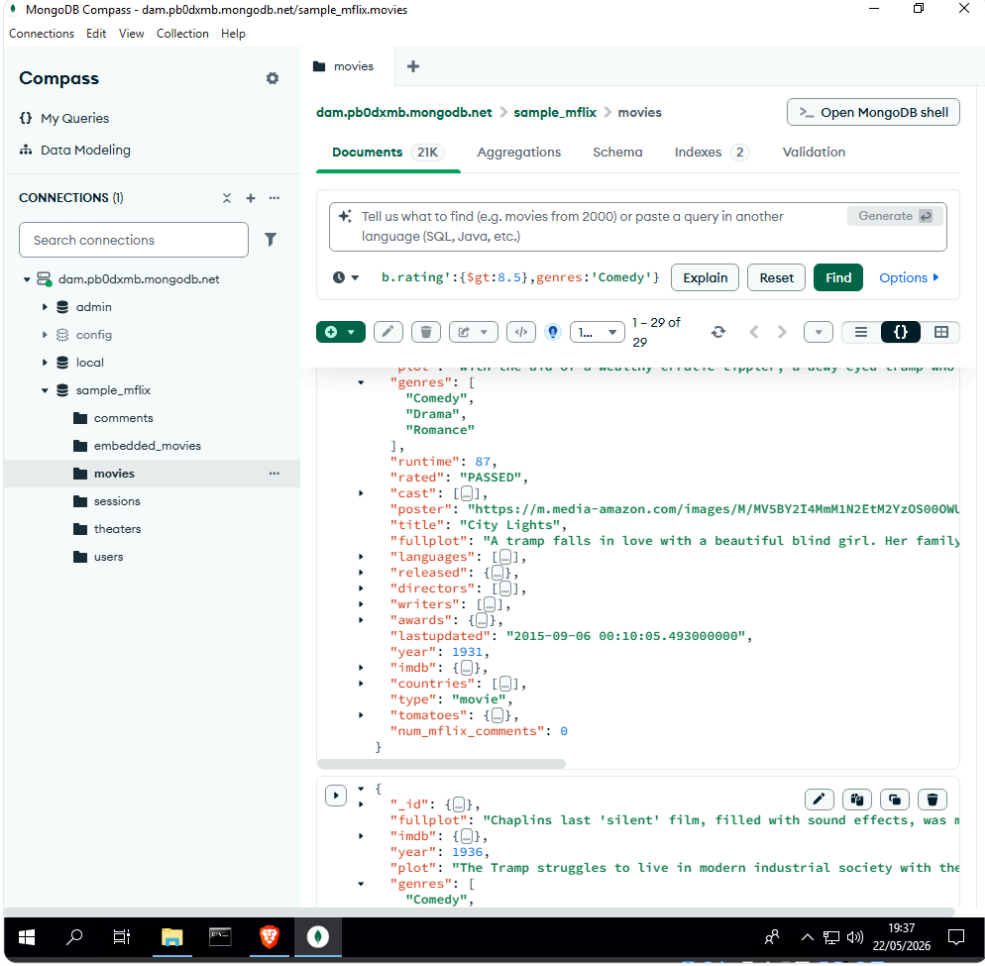
- **C6: Películas que tengan la palabra “matrix” en el título (sin importar mayúsculas/minúsculas).**

```
{ title: { $regex: "matrix", $options: "i" } }
```

Además de filtrar, podemos usar las opciones de **Sort** (por ejemplo, { "imdb.rating": -1 } para ordenar de mejor a peor nota) y **Limit** (por ejemplo, 10 para quedarnos solo con las 10 primeras películas), simulando el ordenamiento y límite de SQL.

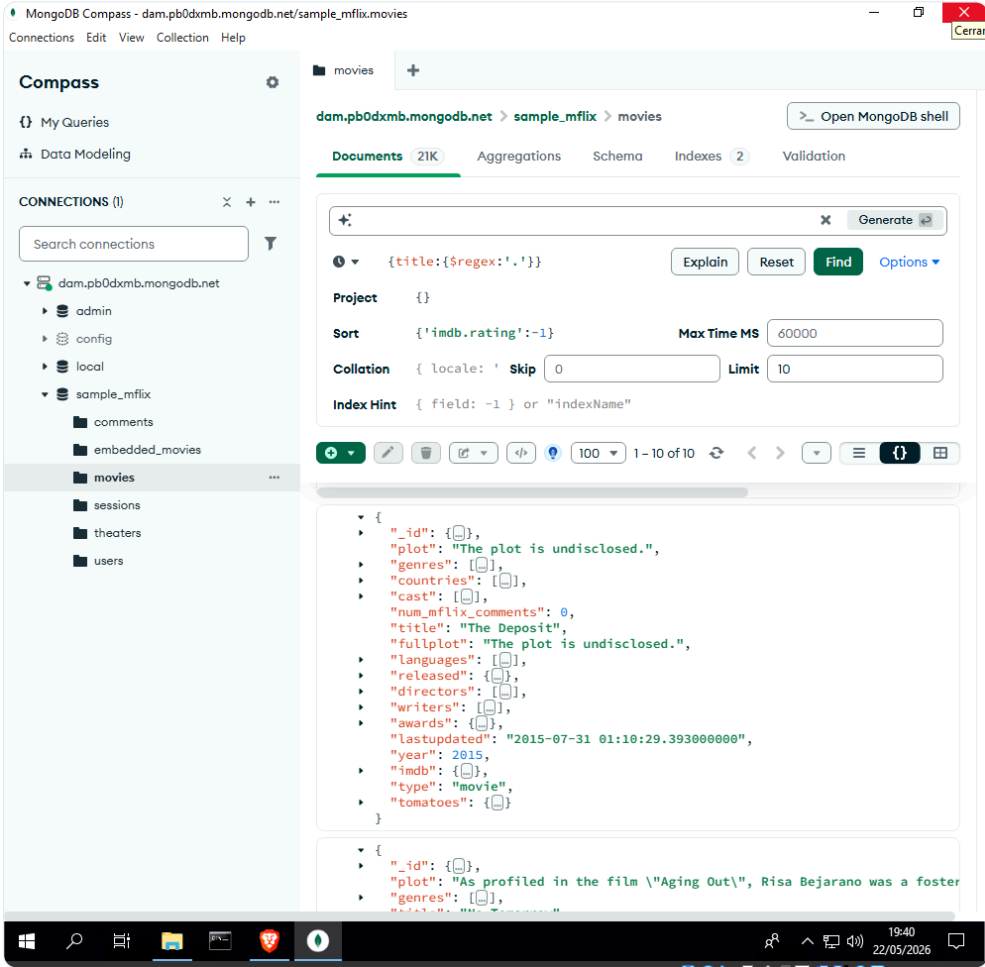


Filtro por año y notas en Compass



The screenshot shows the MongoDB Compass interface. On the left, the 'CONNECTIONS' panel lists the database 'dam.pb0dymb.mongodb.net' and the collection 'sample_mflix.movies'. The main panel displays the 'Documents' tab for the 'movies' collection. A query filter is applied: `b.rating:{$gt:8.5},genres:'Comedy'`. The results show a list of movies, with the first one expanded to show its full document structure, including fields like 'genres', 'runtime', 'rated', 'cast', 'poster', 'title', 'fullplot', 'languages', 'released', 'directors', 'writers', 'awards', 'lastupdated', 'year', 'imdb', 'countries', 'type', 'tomatoes', and 'num_mflix_comments'.

Consulta avanzada con arrays en Compass

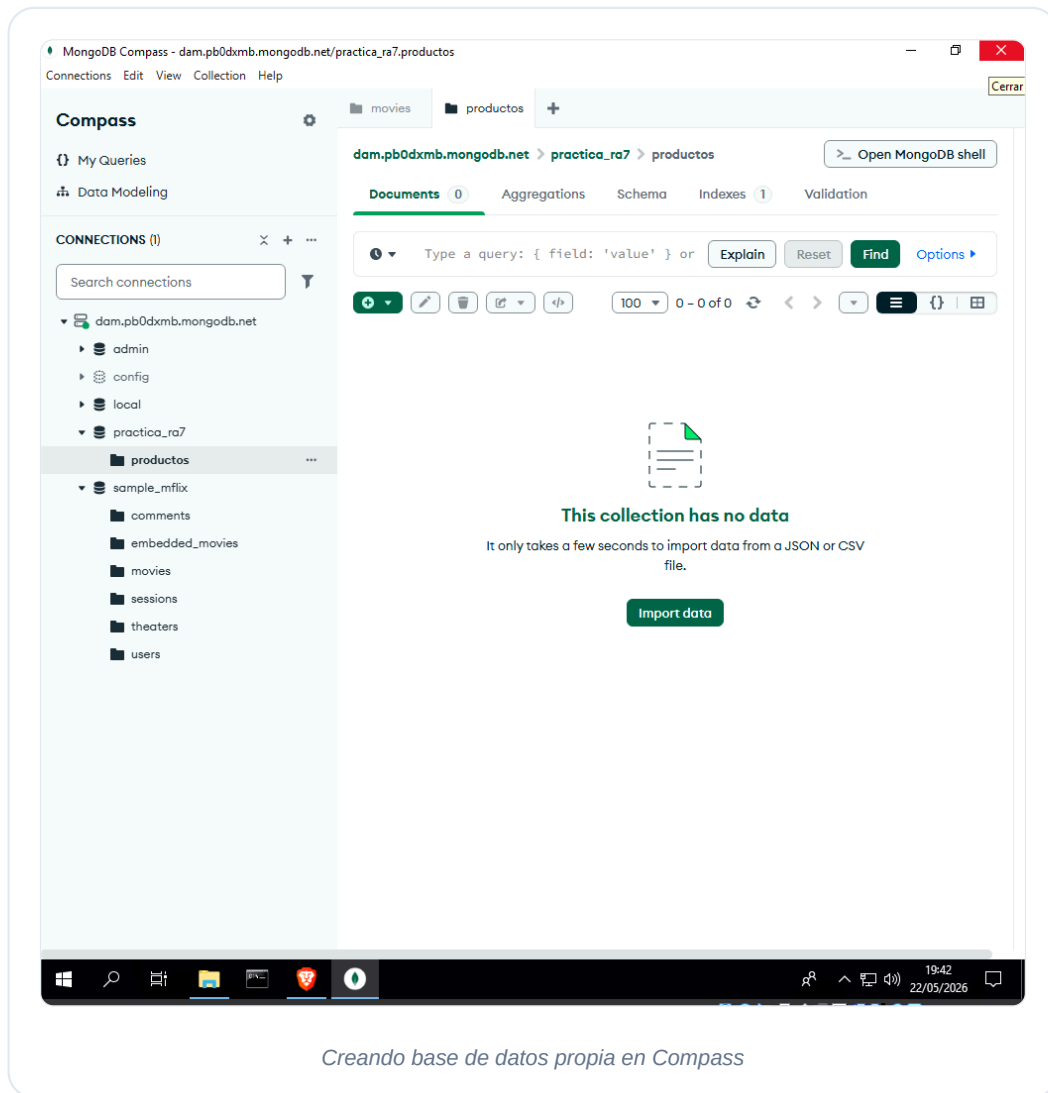


The screenshot shows the MongoDB Compass interface. On the left, the 'CONNECTIONS' panel lists the database 'dam.pb0dxmb.mongodb.net' and the collection 'sample_mflix.movies'. The main panel displays the 'movies' collection with a search query: `{title:{$regex:'.', '}}`. The query is sorted by `{imdb.rating:-1}` with a limit of 10. The results show two movie documents. The first document is for 'The Deposit' (2015), and the second is for 'Aging Out' (2019). The interface includes a search bar, a query editor, and a results pane.

Búsqueda con expresión regular y ordenación

6.4 Creación de una base de datos propia para escritura

Como las bases de datos de ejemplo (`sample_*`) son compartidas y de solo lectura para evitar que las borremos sin querer, decidimos crear nuestra propia base de datos llamada **practica_ra7** con una colección llamada **productos** para poder probar operaciones de escribir, cambiar y borrar.

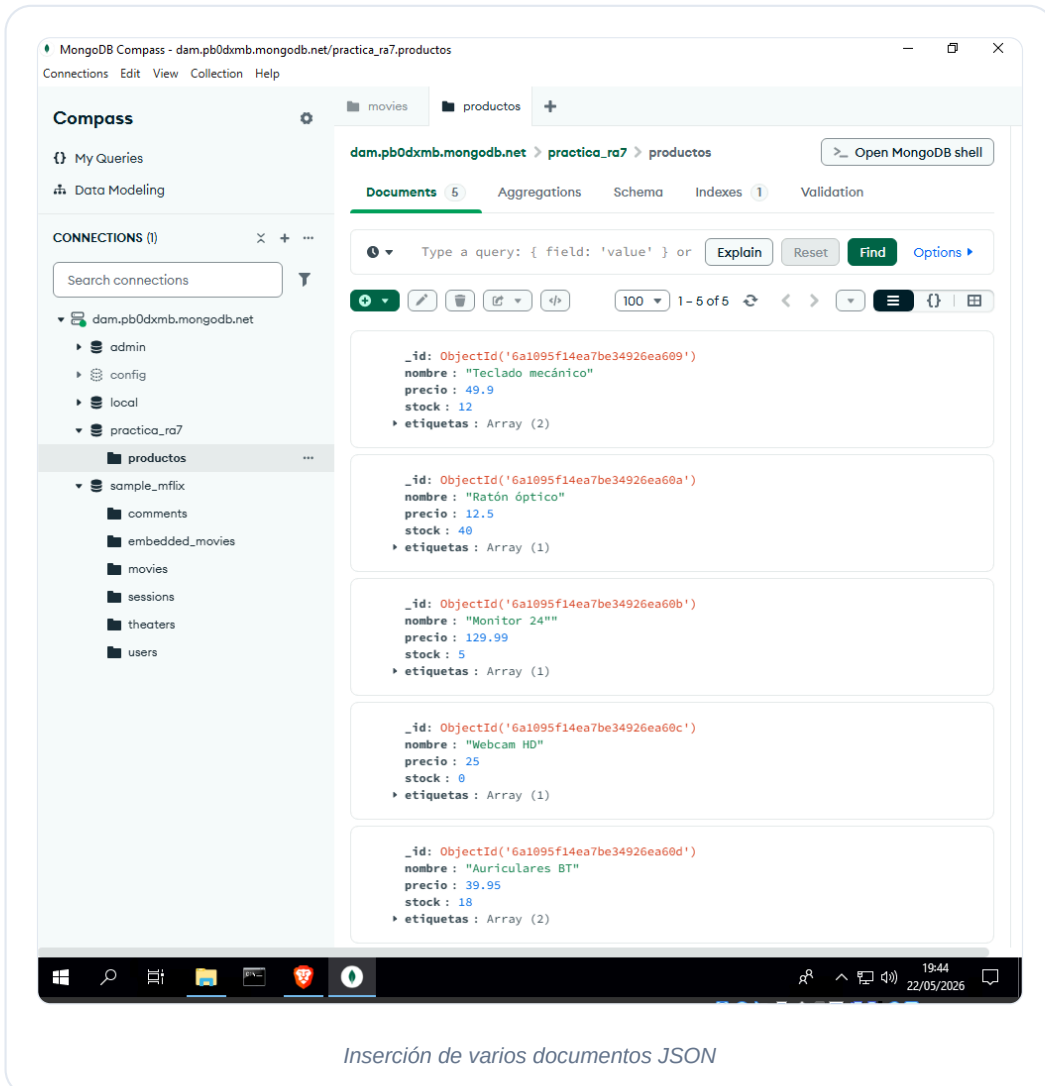


6.5 Inserción de documentos (CREATE)

Entramos en nuestra colección `practica_ra7.productos`, pulsamos en **Add Data -> Insert Document** y pegamos el siguiente array de JSON con 5 productos informáticos listos para vender. Compass detecta que es una lista de documentos y los inserta todos a la vez.

```
[
  { "nombre": "Teclado mecánico", "precio": 49.90, "stock": 12, "etiquetas":
    ["gaming", "USB"] },
  { "nombre": "Ratón óptico", "precio": 12.50, "stock": 40, "etiquetas":
    ["USB"] },
  { "nombre": "Monitor 24\"", "precio": 129.99, "stock": 5, "etiquetas":
    ["oficina"] },
  { "nombre": "Webcam HD", "precio": 25.00, "stock": 0, "etiquetas":
    ["videollamada"] },
  { "nombre": "Auriculares BT", "precio": 39.95, "stock": 18, "etiquetas":
```

```
["audio", "bluetooth"] }
]
```



6.6 Actualización (UPDATE) y borrado (DELETE) desde Compass

Compass nos permite modificar y eliminar documentos de forma gráfica, sin escribir comandos:

- **Modificar (Update):** Buscamos la webcam HD (que tenía stock 0), pulsamos en el icono del **lápiz** que sale al pasar el ratón por encima, cambiamos el stock a 20 y pulsamos en **Update**.
- **Eliminar (Delete):** Buscamos el Ratón óptico, pulsamos sobre el icono de la **papelera** y confirmamos el borrado.

MongoDB Compass - dam.pb0dxmb.mongodb.net/practica_ra7.productos

Connections Edit View Collection Help

Compass

My Queries

Data Modeling

CONNECTIONS (1)

Search connections

dam.pb0dxmb.mongodb.net

- admin
- config
- local
- practica_ra7
 - productos
- sample_mflix
 - comments
 - embedded_movies
 - movies
 - sessions
 - theaters
 - users

dam.pb0dxmb.mongodb.net > practica_ra7 > productos

Documents 5 Aggregations Schema Indexes 1 Validation

Type a query: { field: 'value' } or Explain Reset Find Options

100 1 - 5 of 5

precio : 49.9
stock : 12
etiquetas : Array (2)

_id: ObjectId('6a1895f14ea7be34926ea60a')
nombre : "Ratón óptico"
precio : 12.5
stock : 48
etiquetas : Array (1)

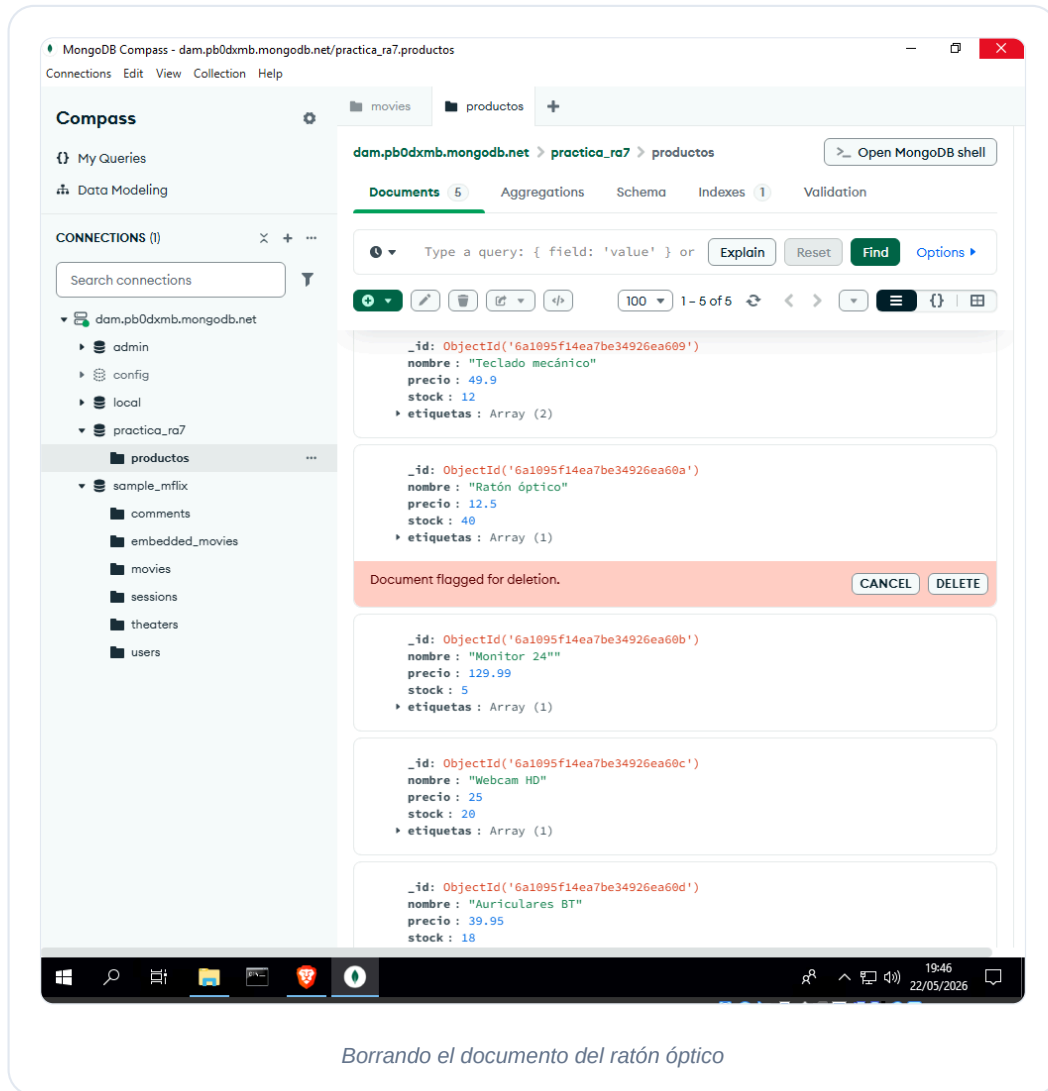
_id: ObjectId('6a1895f14ea7be34926ea60b')
nombre : "Monitor 24"
precio : 129.99
stock : 5
etiquetas : Array (1)

1 _id: ObjectId('6a1895f14ea7be34926ea60c') ObjectId
2 nombre : "Webcam HD" String
3 precio : 25 Int32
4 stock : 28 Int32
5 etiquetas : Array (1) Array

Document modified. CANCEL UPDATE

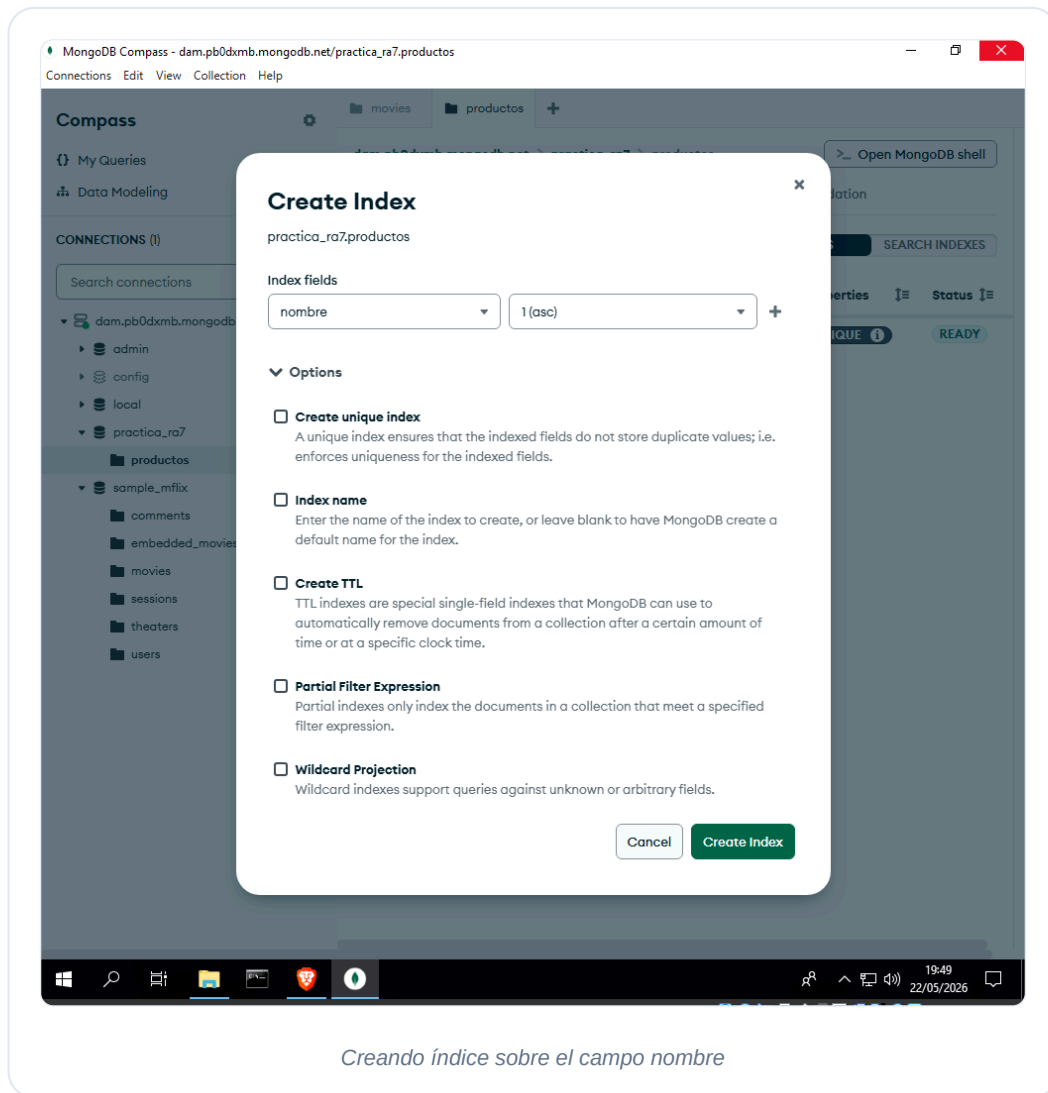
_id: ObjectId('6a1895f14ea7be34926ea60d')
nombre : "Auriculares BT"
precio : 39.95
stock : 18
etiquetas : Array (2)

Modificando el stock de la webcam



6.7 Creación de Índices

Por defecto, MongoDB solo indexa el campo `_id`. Para que las búsquedas por el nombre de los productos vayan mucho más rápidas en el futuro, fuimos a la pestaña **Indexes**, pulsamos en **Create Index**, le pusimos de nombre `nombre_1`, seleccionamos el campo `nombre` y elegimos el tipo `1` (orden ascendente).



6.8 Pipeline de Agregación (Consulta avanzada)

Volvemos a `sample_mflix.movies` para hacer una consulta estadística compleja. Queremos calcular **cuántas películas hay y cuál es su nota media para cada género, teniendo en cuenta solo películas del año 2000 en adelante.**

En la pestaña **Aggregations** de Compass, fuimos añadiendo las siguientes etapas una a una:

1. `\$match`: Filtramos las películas que sean del año 2000 o posterior y que tengan nota registrada en IMDb.

```
{ year: { $gte: 2000 }, "imdb.rating": { $exists: true } }
```

2. `\$unwind`: Como una película puede pertenecer a varios géneros a la vez (están en un array), usamos `\$unwind` para “desenrollar” la lista y duplicar la película para cada género individual.

```
"$genres"
```

3. `\$group`: Agrupamos por género (`\$genres`), contamos cuántas hay sumando 1 por cada una y calculamos la media del campo `imdb.rating`.

```
{ _id: "\$genres", peliculas: { \$sum: 1 }, ratingMedio: { \$avg: "\$imdb.rating" } }
```

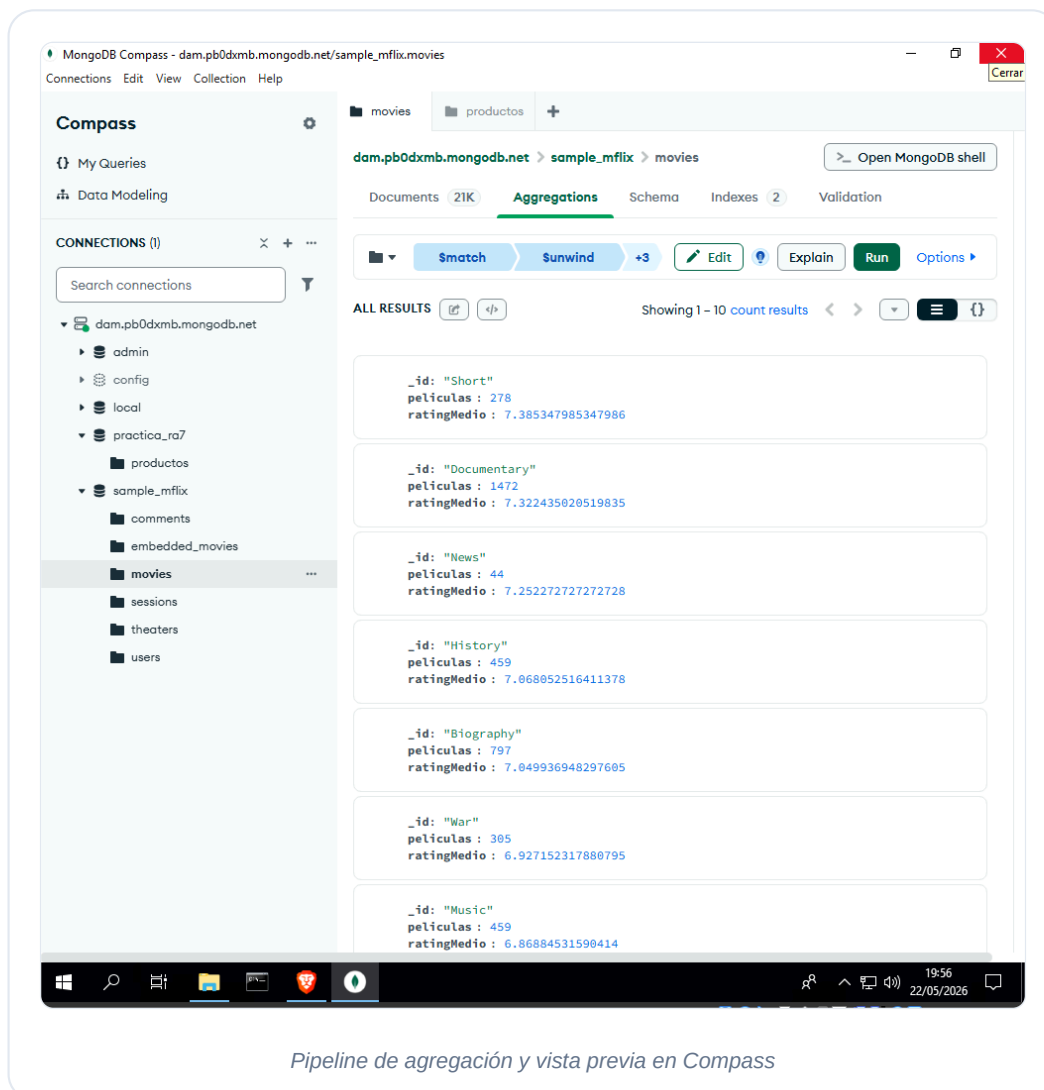
4. `\$sort`: Ordenamos los resultados de mayor a menor nota media.

```
{ ratingMedio: -1 }
```

5. `\$limit`: Nos quedamos solo con los 10 géneros con mejor nota.

```
10
```

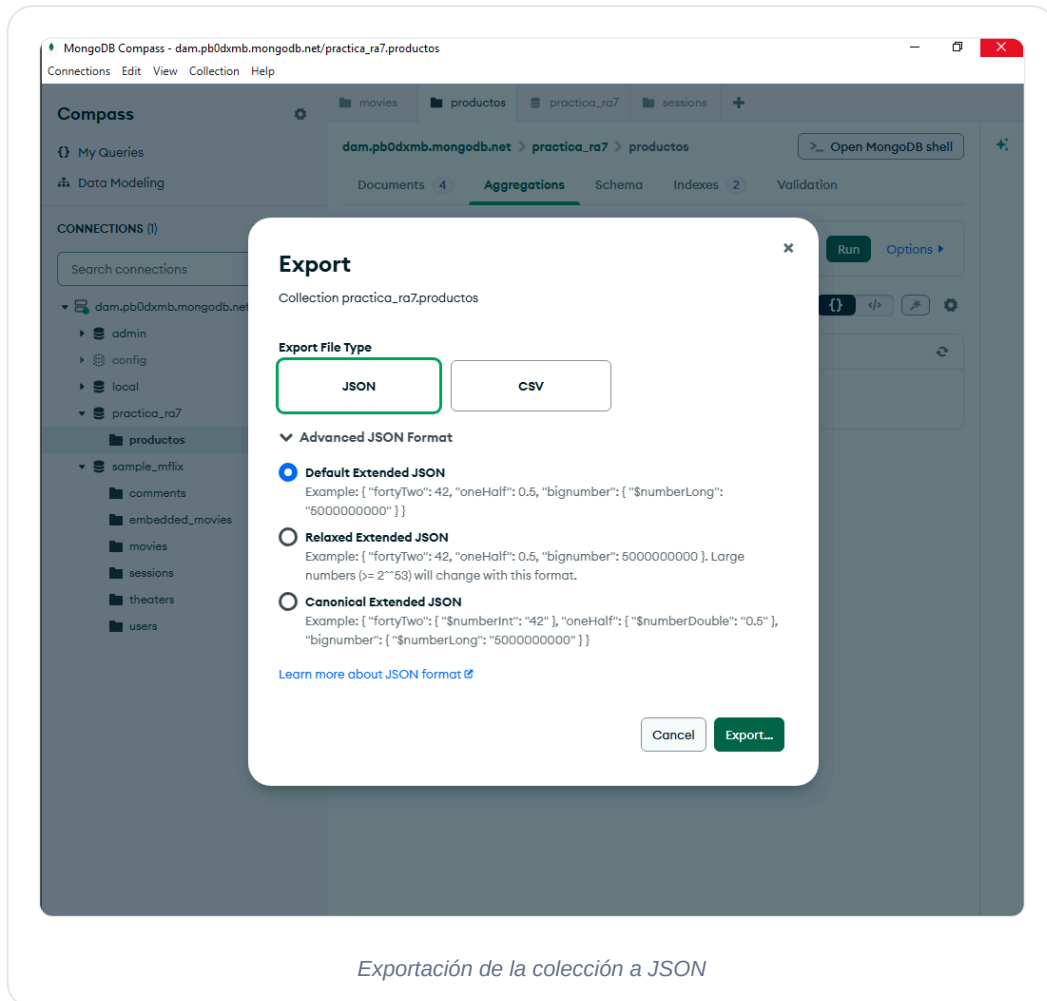
Compass mola mucho para esto porque en cada etapa que añades te va enseñando una vista previa de cómo se van transformando los datos en tiempo real.

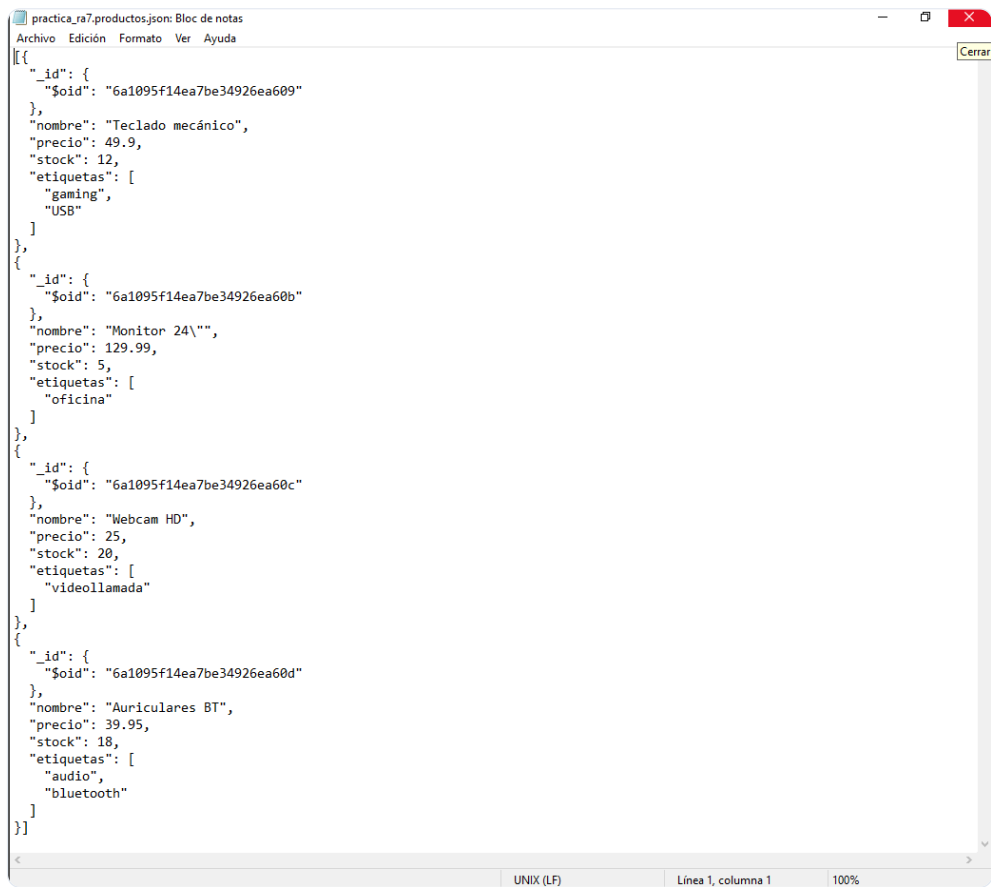


6.9 Exportación e importación de datos

Una tarea básica de administración es sacar datos e introducirlos en colecciones:

- **Exportar:** Entramos en nuestra colección productos, pulsamos en el botón **Export Data**, elegimos exportar todos los campos en formato JSON y guardamos el fichero.
- **Importar:** Para probar que el archivo funciona, nos creamos una colección vacía llamada productos_copia, pulsamos en **Import Data**, seleccionamos nuestro archivo JSON exportado y vemos cómo se cargan todos los productos idénticos en un segundo.

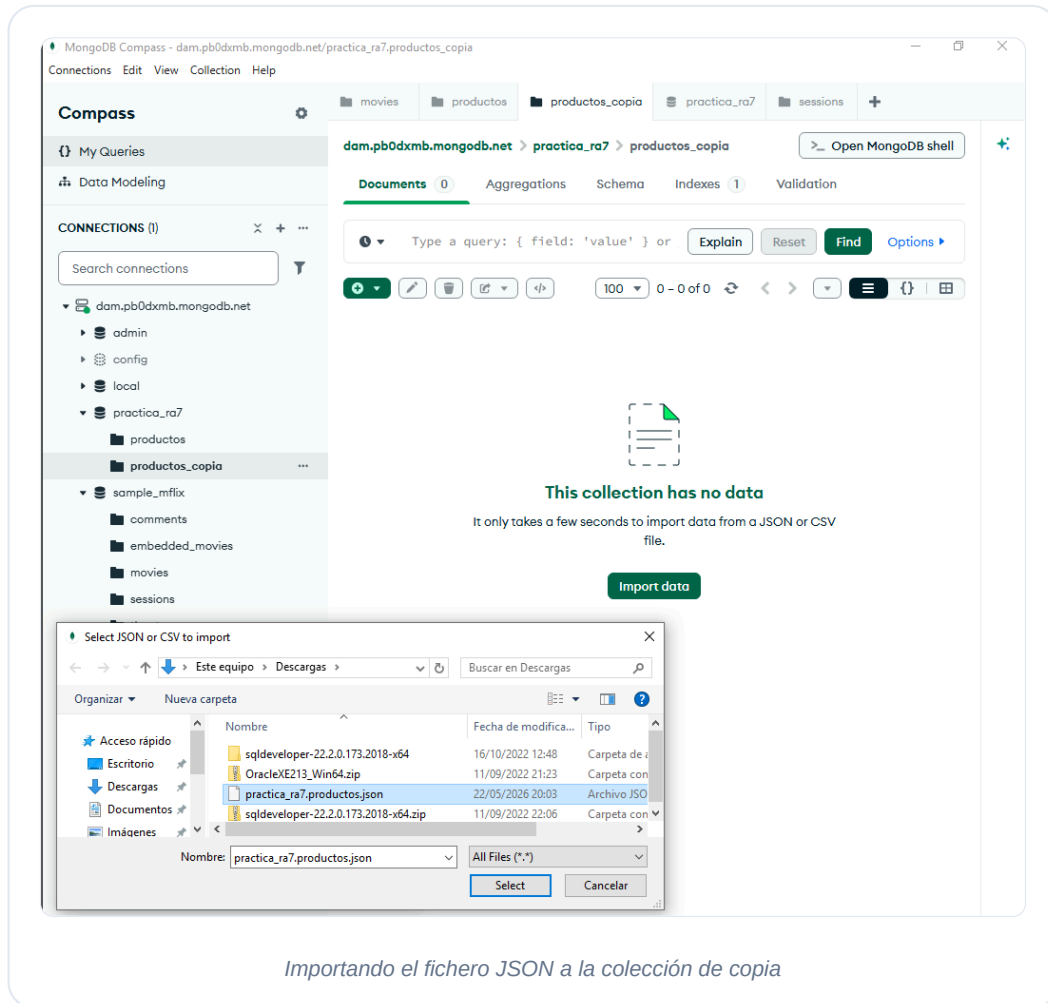




```
[{"_id": {"$oid": "6a1095f14ea7be34926ea609"}, "nombre": "Teclado mecánico", "precio": 49.9, "stock": 12, "etiquetas": ["gaming", "USB"]}, {"_id": {"$oid": "6a1095f14ea7be34926ea60b"}, "nombre": "Monitor 24\"", "precio": 129.99, "stock": 5, "etiquetas": ["oficina"]}, {"_id": {"$oid": "6a1095f14ea7be34926ea60c"}, "nombre": "Webcam HD", "precio": 25, "stock": 20, "etiquetas": ["videollamada"]}, {"_id": {"$oid": "6a1095f14ea7be34926ea60d"}, "nombre": "Auriculares BT", "precio": 39.95, "stock": 18, "etiquetas": ["audio", "bluetooth"]}]]
```

UNIX (LF) Línea 1, columna 1 100%

Configuración del archivo de exportación



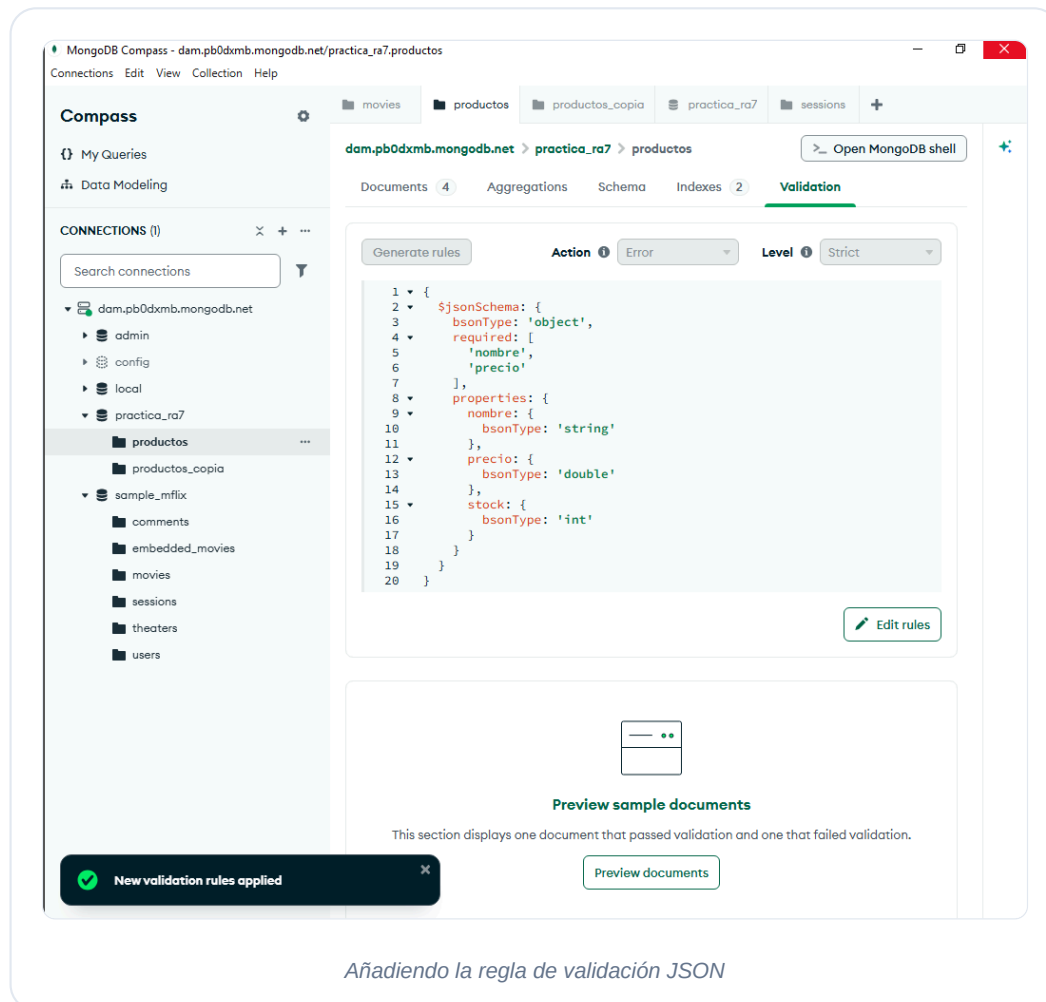
6.10 Validación de esquemas

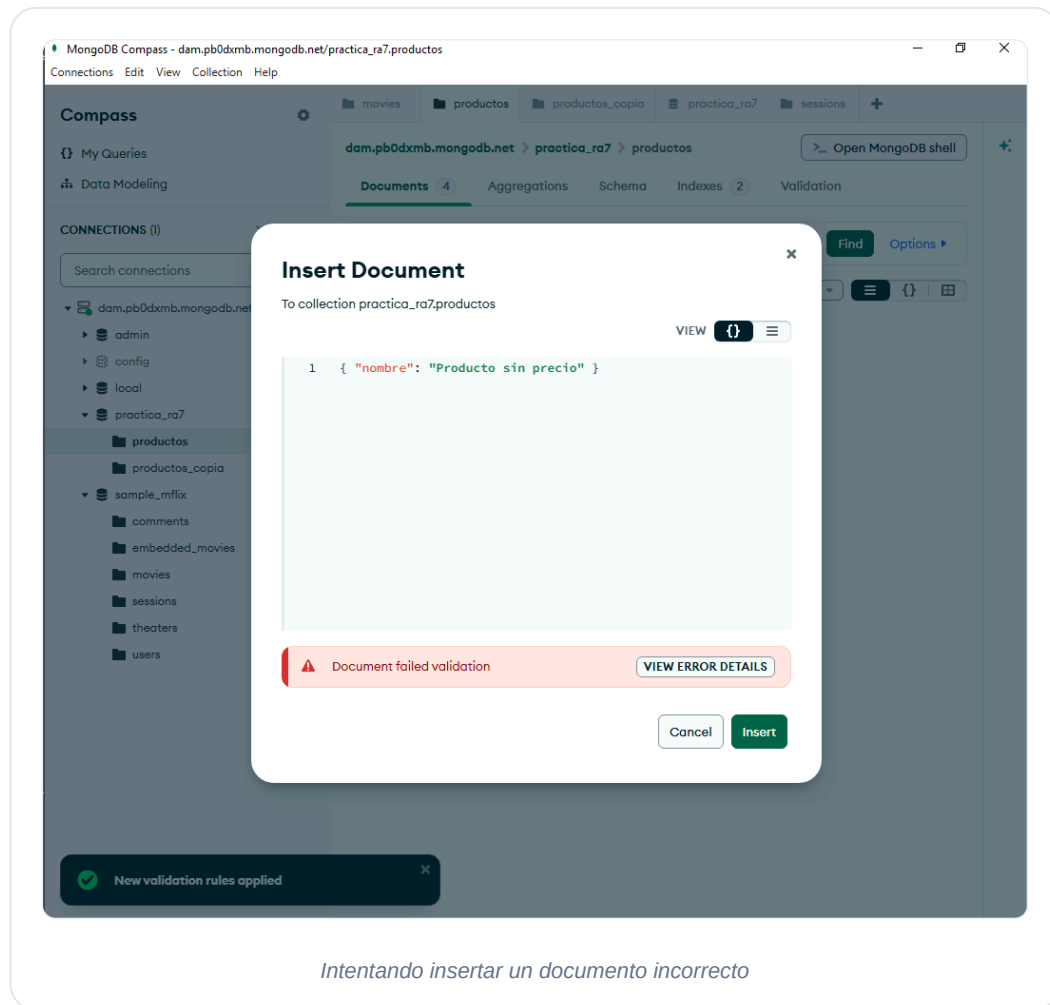
Aunque MongoDB destaca por no obligar a tener un esquema rígido, en la vida real a veces queremos evitar que los usuarios metan tonterías (como un producto sin precio o un precio que sea un texto).

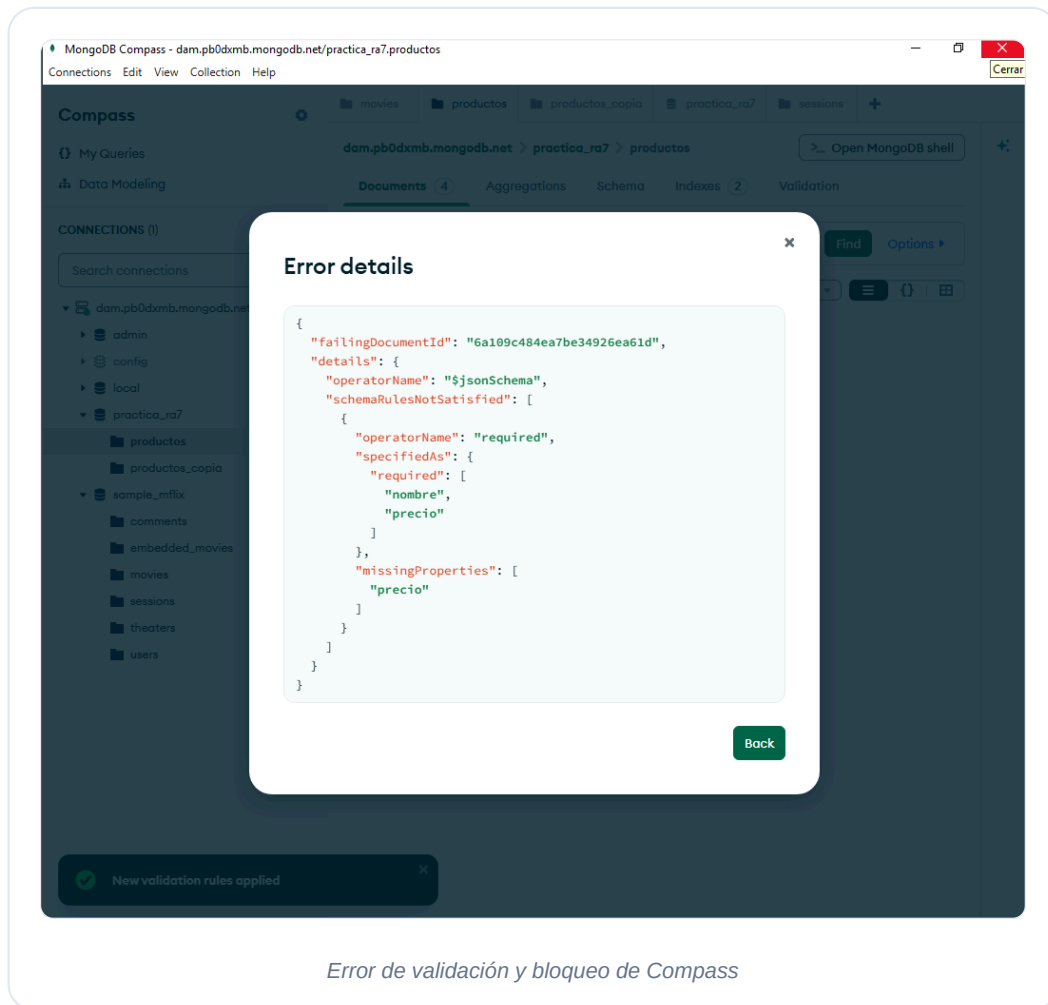
Para configurar reglas, fuimos a nuestra colección `productos`, entramos en la pestaña **Validation**, pulsamos en **Add Rule** y pegamos esta regla en formato `\$jsonSchema`:

```
{
  "\$jsonSchema": {
    "bsonType": "object",
    "required": ["nombre", "precio"],
    "properties": {
      "nombre": { "bsonType": "string" },
      "precio": { "bsonType": "double" },
      "stock": { "bsonType": "int" }
    }
  }
}
```

Configuramos la acción en caso de fallo como **Error** (bloquear la inserción). Tras guardar la regla, probamos a insertar a propósito un producto sin el campo `precio`. Compass nos mostró inmediatamente un mensaje de error diciendo que el documento no cumple las normas de validación y bloqueó la inserción.







7 Conclusiones

Tras realizar este trabajo y trastear a fondo con MongoDB Atlas y Compass, nos llevamos varias ideas clave:

- **No vienen a sustituir a SQL:** Las bases de datos NoSQL no son mejores ni peores que las relacionales, simplemente sirven para problemas diferentes. Si estás haciendo una app de un banco donde cada céntimo debe cuadrar al instante, usa relacional. Si estás haciendo un catálogo de productos de una tienda que cambia cada semana o necesitas guardar miles de registros de sensores por segundo, NoSQL es mucho más cómoda y rápida.
- **La flexibilidad es una gozada pero tiene peligro:** Que MongoDB no te obligue a definir esquemas te permite programar y hacer cambios súper rápido. El problema es que, si no tienes cuidado o no pones reglas de validación en tu código, tu base de datos puede acabar llena de datos desordenados y erróneos.
- **El cambio de mentalidad:** Al principio cuesta entender cómo guardar datos sin hacer JOINS. Sin embargo, cuando te acostumbras a meter arrays y subdocumentos dentro de un mismo registro, te das cuenta de que el diseño es mucho más natural y se parece un montón a cómo programamos con objetos en Java.

- **MongoDB Atlas + Compass es un combo perfecto:** Poder conectarte a una base de datos en la nube sin configurar servidores locales complicados y tener un programa visual para hacer consultas, agregaciones complejas y validaciones con unos pocos clics facilita muchísimo el aprendizaje en primero de DAM.

8 Reparto de tareas

Lo hemos hecho juntos.

9 Bibliografía

- **Documentación oficial de MongoDB:** <https://www.mongodb.com/docs/>
- **MongoDB Atlas (Panel de control en la nube):** <https://www.mongodb.com/atlas>
- **MongoDB Compass (Herramienta gráfica de escritorio):** <https://www.mongodb.com/products/tools/compass>
- **Bases de datos de ejemplo de Atlas:** <https://www.mongodb.com/docs/atlas/sample-data/>
- **Apuntes de Bases de Datos (Campus Virtual), Unidad de Aprendizaje 7 (UT7):** Gestión de bases de datos no relacionales.
- **Asistente de Inteligencia Artificial integrado en MongoDB Compass** (usado para resolver dudas puntuales de sintaxis MQL durante las pruebas).
- **Videotutorial de MongoDB para principiantes (conceptos y primeros pasos):** https://www.youtube.com/watch?v=nIOWsnO-d7Q&list=PLXXiznRYETLcJE_4U9qN2pysZOSYyL4Mh